

GenAI Interview Master Guide

Complete Q&A Reference

RAG · LangGraph · LLMs · Vector DBs · EY Scenario Patterns

*Compiled from: Infosys Interview Notes + EY Interview Experience
Narayanan | Accenture | September 2026*

Contents

Section 1	RAG Deep Dive — Infosys Notebook	<i>12 Questions</i>
Section 2	GenAI Fundamentals — Infosys Notebook 2	<i>10 Questions</i>
Section 3	EY Scenario-Based Interview	<i>9 Scenarios</i>
Section 4	EY vs Infosys — Pattern Analysis	<i>Comparison</i>
Section 5	Quick-Reference Cheat Sheet	<i>Key Formulas</i>
Section 6	Square Yards — ML · Python · DSA · HR	<i>13 Questions</i>
Section 7	FAANG AI Engineer Cheat Sheet — All 15 Topics	<i>90+ Concepts</i>
Section 8	AI/ML Interview — ML Models · OCR · RAG Follow-ups	<i>14 Questions</i>
Section 9	Agentic AI Engineer (FinServ) — Inference · KV Cache · Distillation · RAG Ops	<i>14 Questions</i>

SECTION 1 — RAG Deep Dive | Infosys Interview Questions

Q1 [Infosys]

Difference between top-p and top-k?

top-k: At each generation step, consider only the k most probable next tokens. Hard cutoff — rank 51+ tokens are discarded regardless of their probability.

top-p (nucleus sampling): Consider the smallest set of tokens whose cumulative probability $\geq p$. Dynamic cutoff — adapts to the model's confidence distribution.

Parameter	Behaviour	Use case
top-k = 50	Always picks from top 50 tokens	Predictable, fast outputs
top-p = 0.9	Picks tokens covering 90% probability mass	Natural, adaptive diversity
Both combined	top-k first, then top-p prunes further	Production default — best of both

■ Interview Tip

Most production LLM calls use both: top-k=50 + top-p=0.9 + temperature=0.7. Know which knob to turn — temperature controls randomness, top-p controls vocabulary breadth.

Q2 [Infosys]

If chunk size is too small and too large, what will be the issue?

	Too Small (e.g. 64 tokens)	Too Large (e.g. 2048 tokens)
Embedding quality	Poor — sparse semantic signal	Diluted — one vector can't represent 2k tokens
Retrieval precision	High recall, low precision	Low recall on specific queries
Context noise	Many chunks needed, high overhead	Relevant + irrelevant content mixed
LLM cost	More chunks = more lookups	Larger context window usage per call

Sweet spot: 256–512 tokens for dense prose, 128 for Q&A. Always add overlap of 10–20%.

Q3 [Infosys]

What are the different types of document loaders?

Common loaders and when to use them:

- **PyPDFLoader / PDFPlumber** — text-based PDFs, clean layout
- **UnstructuredFileLoader** — mixed content: PDF, DOCX, HTML, images
- **AzureAIDocumentIntelligenceLoader** — scanned PDFs, tables, complex forms
- **CSVLoader / JSONLoader** — structured tabular or JSON data
- **WebBaseLoader** — HTML scraping from URLs
- **DirectoryLoader** — batch load entire folders

Decision rule: scanned or complex layout → Azure DI / Unstructured. Clean text PDF → PyPDF is sufficient.

Q4 [Infosys]

What are the different types of text splitters and which do you use in your project?

Splitter	Split Strategy	Best For
RecursiveCharacterTextSplitter	Paragraph → sentence → word hierarchy	Default — general prose (recommended)
CharacterTextSplitter	Single delimiter (e.g. newline)	Simple, homogeneous text
TokenTextSplitter	Hard token count limit	When LLM context window is the constraint
MarkdownHeaderTextSplitter	Splits on # headers	Structured docs, wikis, README files
SemanticChunker	Similarity-based boundary detection	Highest quality, slowest — offline pipelines
HTMLSectionSplitter	HTML semantic tags	Web content with meaningful structure

Project answer template: "I use RecursiveCharacterTextSplitter with chunk_size=512 and chunk_overlap=50 because it respects natural language boundaries and is production-reliable. For legal/policy documents with section headers, I switch to MarkdownHeaderTextSplitter to preserve structural metadata as retrieval filters."

■ Interview Tip

Always justify your choice with a project reason, not just the definition. Interviewers reward 'why' over 'what'.

Q5 [Infosys]

How to check/evaluate accuracy of a RAG application?

Four core RAGAS metrics:

Metric	What it measures	Catches
Context Precision	Of retrieved chunks, how many were relevant?	Over-retrieval / noise
Context Recall	Of all relevant content, how much was retrieved?	Under-retrieval / missed chunks
Faithfulness	Does the answer use only retrieved context?	Hallucination
Answer Relevancy	Is the answer on-topic for the question?	Off-topic generation

Tooling: RAGAS library auto-computes all four. For production: log queries + retrieved chunks + answers → periodic LLM-as-a-judge evaluation pipeline.

Q6 [Infosys]

How to optimize tokens?

- **Trim chunks** — pass only the most relevant portion, not the entire chunk
- **Similarity threshold filter** — discard chunks below cosine similarity 0.75 before sending to LLM
- **Context compression** — summarize retrieved chunks before final generation (map-reduce chain)
- **max_tokens cap** — always set on LLM output to prevent runaway generation
- **Cache repeated prompts** — semantic caching with GPTCache or Azure prompt cache
- **Right-size the model** — use GPT-3.5 / smaller models for simple tasks, GPT-4o only for complex reasoning

Q7 [Infosys]

How to customize output of LLM?

- **System prompt** — define role, tone, format constraints, and output rules
- **Output parsers** — PydanticOutputParser, JsonOutputParser (LangChain) enforce structured schema
- **Few-shot examples** — show 2–3 examples of the exact format you want in the prompt
- **Temperature** — lower (0.0–0.3) = deterministic; higher (0.7–1.0) = creative
- **Stop sequences** — halt generation at a specific token (e.g. stop=["\n", "END"])
- **Structured outputs** — OpenAI response_format={"type": "json_object"} or function calling

Q8 [Infosys]**Full form of RAG and what is 'Augmented'?****RAG = Retrieval-Augmented Generation**

The '**Augmented**' refers to the **prompt** being augmented — the user's raw query is enriched with retrieved context documents before being sent to the LLM. The model itself is NOT fine-tuned or modified; only the input is augmented.

Flow: User query → embed → vector search → retrieve top-k chunks → inject into prompt → LLM generates grounded answer.

■ Interview Tip

Many candidates say 'the model is augmented' — that's wrong. The prompt is augmented. This distinction often comes up in follow-up questions.

Q9 [Infosys]**How to solve partial failure of LangGraph application?**

Five-layer resilience strategy:

1. **Checkpointing** — LangGraph's MemorySaver / SqliteSaver persists state after each node. On failure, resume from last checkpoint, not from scratch.
2. **Node-level try/except** — each node catches its own exceptions and returns an error state rather than crashing the full graph.
3. **Conditional edges** — route to a fallback node if a node returns error state (e.g. retrieval_failed → use_cached_response).
4. **Retry logic** — wrap LLM calls with tenacity (exponential backoff). Critical for transient API errors.
5. **Dead-letter state** — if a node fails N times, transition to 'human_review' or 'graceful_degradation' state instead of infinite retry.

Q10 [Infosys]**What is state in LangGraph?**

State is a **typed dictionary (TypedDict)** that flows through every node in the graph. Each node reads from state and writes back to state. It is the single source of truth for the entire workflow.

Example state schema:

```
class AgentState(TypedDict):
    query: str
    retrieved_docs: list[Document]
    answer: str
    error: Optional[str]
    retry_count: int
```

State enables: partial failure recovery (checkpointing), conditional branching (routing on state values), and observability (log state at each node).

Q11 [Infosys]

What is chunk overlap and why is it important?

Chunk overlap means the last N tokens of chunk[i] are repeated as the first N tokens of chunk[i+1].

Why it matters: If a key sentence falls exactly at the boundary between two chunks, without overlap it is split — neither chunk has full context. With overlap=50 tokens, the boundary sentence appears in both chunks, so retrieval catches it regardless of which chunk scores higher.

Recommended overlap: 10–20% of chunk size. For chunk_size=512 → overlap=50–100 tokens.

Q12 [Infosys]

What is dense vs sparse retrieval?

	Dense Retrieval	Sparse Retrieval
Method	Embedding vectors (semantic similarity)	Keyword frequency — BM25, TF-IDF
Strength	Understands meaning, handles paraphrasing	Exact keyword match — fast, no GPU
Weakness	Misses exact terms: acronyms, codes, IDs	Misses semantic intent / synonyms
Example query	'What causes chest pain?' → finds 'cardiac symptoms'	'HDFC ULIP clause 4.2' → exact match
Tooling	pgvector, Pinecone, Weaviate, Chroma	Elasticsearch BM25, Whoosh, LangChain BM25Retriever

Production standard: Hybrid retrieval (dense + sparse), results merged with Reciprocal Rank Fusion (RRF), then re-ranked with a cross-encoder. This is why BM25 alongside pgvector is the right architecture.

■ Interview Tip

If asked 'why BM25 alongside vector retrieval?' — they cover each other's blind spots. Dense misses exact codes; sparse misses semantic intent. Together they are more robust.

SECTION 2 — GenAI Fundamentals | Infosys Notebook 2

Q1 [Infosys]

What is hallucination, throttling, and rate limiting?

Hallucination: LLM generates confident but factually wrong content. Caused by gaps in training data, over-reliance on pattern completion, or lack of grounding. Fix: RAG (ground in retrieved facts) + faithfulness evaluation.

Throttling: Intentional slowing of request processing by the server when load is high. Server-side protection mechanism — not an error, but a signal to back off.

Rate Limiting: Hard cap on requests per minute/hour per API key. Exceeding it returns HTTP 429. Fix: exponential backoff with jitter, request queuing, or multiple API key rotation.

Q2 [Infosys]

How to control output of LLM?

- **System prompt** — role definition, tone, format rules, constraints ('respond only in JSON')
- **Output parsers** — PydanticOutputParser enforces typed schema; JsonOutputParser for dict output
- **Few-shot examples** — demonstrate exact format in 2–3 examples before the actual query
- **Temperature / top-p / top-k** — control randomness and vocabulary diversity
- **Stop sequences** — halt generation at a defined token to prevent over-generation
- **Structured outputs (OpenAI)** — `response_format={"type": "json_object"}` or tool/function calling

Q3 [Infosys]

What is RAG and how to implement?

RAG = Retrieval-Augmented Generation: retrieve relevant documents at query time → inject into prompt → generate grounded answer. Avoids fine-tuning by keeping knowledge external and updateable.

Implementation steps:

1. Chunk documents → generate embeddings → store in vector DB
2. At query time: embed query → similarity search → retrieve top-k chunks

3. Build prompt: system instructions + retrieved chunks + user question
4. Call LLM → return grounded answer with source citations
5. Evaluate: faithfulness + relevancy + context recall (RAGAS)

Q4 [Infosys]

Architecture of Transformer Model?

Input → **Tokenization** → **Embedding + Positional Encoding** → $N \times$ [**Multi-Head Self-Attention** → Add & Norm → **Feed-Forward Network** → Add & Norm] → **Output Projection** → Softmax

Key insight: Self-attention allows every token to attend to every other token in the sequence simultaneously — that's what makes transformers powerful vs. RNNs (sequential, limited memory).

Positional encoding: Transformers have no inherent sense of order — positional encodings inject position information into embeddings.

Q5 [Infosys]

What is vector DB? What is indexing in vector DB? What is similarity search?

Vector DB: Stores high-dimensional embedding vectors (e.g., 1536-dim for ada-002) and enables fast approximate nearest-neighbour search. Examples: pgvector, Pinecone, Weaviate, Chroma.

Indexing: Organises vectors so you don't brute-force compare all N vectors at search time. Key algorithms:

- **HNSW** (Hierarchical Navigable Small World) — graph-based, fast, high recall, used by pgvector/Weaviate
- **IVF-Flat** (Inverted File) — clusters vectors, searches only relevant clusters, used by Faiss/Pinecone

Similarity search: Finds the k nearest vectors to a query vector using:

- **Cosine similarity** — angle between vectors (most common for text, scale-invariant)
- **Dot product** — magnitude + direction (faster, used when embeddings are normalised)
- **L2 distance** — Euclidean distance (used for image embeddings)

Q6 [Infosys]

How to evaluate LLM output?

Three evaluation approaches:

1. **RAGAS metrics** — Context Precision, Context Recall, Faithfulness, Answer Relevancy (automated, LLM-based scoring)
2. **LLM-as-a-judge** — use a separate LLM (e.g., GPT-4o) to score answers on a rubric: accuracy, groundedness, completeness
3. **Human evaluation** — golden test set with expert-labelled ground truth answers; measure BLEU/ROUGE for extractive tasks

Production monitoring: Log query + retrieved chunks + answer + user feedback → periodic batch evaluation → alert on faithfulness drop.

Q7 [Infosys]

How to apply summarization techniques using RAG?

Map-Reduce pattern: Split document into chunks → summarize each chunk independently (map) → combine summaries → generate final summary (reduce).

RAG-enhanced summarization: Retrieve only the relevant chunks before summarizing, rather than processing the entire document. Steps:

1. Retrieve top-k relevant chunks based on query (e.g. 'summarize the key findings')
2. Map: generate a mini-summary for each retrieved chunk
3. Reduce: combine mini-summaries into a coherent final summary
4. Add citations so readers can trace each point back to the source chunk

Q8 [Infosys]

How to use parallel programming in LLM requests?

Use **asyncio** with **asyncio.gather()** to fire multiple LLM calls concurrently:

```
import asyncio
from langchain_openai import AzureChatOpenAI

llm = AzureChatOpenAI(...)
sem = asyncio.Semaphore(5) # respect rate limits

async def call_with_limit(chunk):
    async with sem:
        return await llm.ainvoke(chunk)

results = await asyncio.gather(*[call_with_limit(c) for c in chunks])
```

In LangGraph: Nodes that don't depend on each other can run as parallel branches — define them as separate edges from the same source node.

Rate limit guard: Always use a Semaphore to cap concurrent calls to your API quota.

Q9 [Infosys]

What problems did you face in your project and how did you solve them?

Structure your answer as: Situation → Problem → Decision → Result

Example frame for Rapid Diagnostics project:

- **Situation:** Multi-document retrieval across large Azure resource audit logs
- **Problem:** Token limits exceeded when passing all retrieved chunks — context overflow causing truncation
- **Decision:** Added re-ranking step — retrieve top-20, re-rank with cross-encoder, pass only top-5 to LLM
- **Result:** Reduced hallucinations, stayed within 8k context window, improved faithfulness score from 0.71 to 0.89

Adapt this to your actual project experience — the framework is what the interviewer is evaluating, not the specific numbers.

Q10 [Infosys]

What is tokenization?

Tokenization converts raw text into **tokens** (subword units) that the model processes. Most LLMs use **Byte Pair Encoding (BPE)**:

- Frequent subword sequences are merged into single tokens
- 'tokenization' → ['token', 'ization'] (2 tokens)
- 'GPT-4' → ['G', 'PT', '-', '4'] (4 tokens)

Practical rule of thumb: 1 token ≈ 0.75 words in English. So 1000 words ≈ 1333 tokens. Token count ≠ word count — always check with tiktoken before estimating costs.

Why it matters for RAG: Chunk sizes must be set in tokens, not characters. A 'chunk_size=512' means 512 tokens, not 512 characters.

SECTION 3 — EY Scenario-Based Interview | Production Reasoning

EY asks ZERO definitional questions. Every question is: 'X is broken in production. How do you diagnose and fix it?'

Q1 [EY Round 1]

RAG chatbot is confidently giving wrong information not in the knowledge base. Client is angry. Diagnose and fix in production today.

Step 1 — Separate retrieval failure from generation failure first:

Check retrieval scores and log actual chunks being passed to LLM for failing queries.

Two failure modes: (a) wrong chunks retrieved, or (b) correct chunks retrieved but LLM ignores them.

Step 2 — If retrieval is failing:

Lower similarity threshold, improve embeddings, check for metadata filtering issues, add BM25 hybrid retrieval.

Step 3 — If generation is hallucinating (retrieval correct):

Tighten system prompt with explicit grounding: 'Answer ONLY using the provided context. If unsure, say I don't know.'

Add LLM-as-a-judge groundedness check before response reaches user.

Step 4 — Immediate mitigation (before full fix ships):

If groundedness score below threshold → return safe fallback: 'I do not have this information.'
Under-performing visibly is better than confidently wrong.

■ EY Pattern

The diagnostic framework is: Retrieval failure OR Generation failure? Never jump to fix before diagnosing which layer broke.

Q2 [EY Round 1]

Knowledge base grew from 10K to 5M documents. Latency crashed, costs increasing. What do you do?

- **Shard or upgrade vector DB** — move to Pinecone/Weaviate/pgvector with proper HNSW indexing configured for scale

- **Tune ANN index parameters** — trade a small amount of recall for significant speed gain (e.g. ef_search in HNSW)
- **Metadata pre-filtering** — filter by document type, date, department before vector search runs. Shrinks search space from 5M to thousands.
- **Cache frequent queries** — semantic cache for top-N repeated query patterns
- **Tiered retrieval** — cheap BM25 keyword filter first → vector search only on BM25 results. At 5M docs, flat vector search is the wrong architecture.

■ EY Pattern

The senior answer: 'Ask whether 5M documents should even be in a flat index, or whether tiered retrieval is better.' This shows systems thinking, not just tooling knowledge.

Q3 [EY Round 1]

GenAI feature approved at \$2K/month LLM budget, now at \$18K/month and growing. Client demands explanation.

Diagnose first — identify the cost driver:

- **Model size mismatch** — using GPT-4o for tasks a GPT-3.5 or smaller model can handle? Route by task complexity.
- **Prompt size bloat** — sending full documents instead of retrieved chunks? Each extra 1000 tokens = direct cost.
- **Duplicate calls** — retrieval storms or bugs causing the same query to fire multiple times?
- **No caching** — repeated prompts regenerating instead of serving cached responses?
- **Volume spike** — more users or more queries than forecasted? Scale the budget or add throttling.

Fix strategy: Model routing (small model for simple, large for complex) + prompt trimming + semantic caching + duplicate call detection. Implement cost monitoring with alerts at 50% and 80% of budget.

Q4 [EY Round 2]

Design an internal HR policy Q&A; tool: 3,000 PDFs, weekly updates, strict access control (Legal sees only Legal, HR sees only HR). End-to-end.

Layer 1 — Ingestion Pipeline:

- Chunk PDFs, tag each chunk with department metadata at ingestion time
- Delta re-embedding: only re-embed documents that changed weekly, not the full corpus
- Store department tag in vector DB metadata field alongside embedding

Layer 2 — Retrieval with Access Control (most critical):

- Access control must happen AT retrieval time, not as a post-filter on LLM output
- Vector search is scoped by metadata filter: WHERE department = user.department
- NEVER trust the LLM to withhold information — wrong chunks must never reach the prompt

Layer 3 — Generation:

- Grounded answers only, with citations to source document + page number
- System prompt: 'Answer only from provided context. Do not infer or extrapolate.'

Layer 4 — Evaluation & Compliance:

- Small labelled test set per department, checked periodically for accuracy AND access control leakage
- Access control leakage is a compliance issue, not just a UX bug — treat it with severity

■ EY Follow-up

Follow-up answer: For overlapping policies (Legal + HR) — tag document with multiple departments, make access check 'any match' instead of 'exact match'. Never duplicate documents.

Q5 [EY Round 3]

Client insists chatbot should NEVER be wrong. How do you explain this is unrealistic without losing their confidence?

- **Avoid technical jargon entirely** — frame as risk management, not model architecture
- **Reframe the goal:** 'No AI system is 100% accurate — just like humans doing the same work make mistakes. The real question is: how do we detect and contain mistakes?'
- **Point to deliverables, not promises:** guardrails, groundedness checks, fallback responses, human-in-the-loop review for high-stakes queries
- **Show the safety net:** 'We build the system to fail safely — when confidence is low, it says I don't know rather than guessing.'

Q6 [EY Behavioral]

Tell me about a time a project's scope changed midway.

Use **STAR format**: Situation → Task → Action → Result.

Frame for GenAI context: A retrieval pipeline designed for 10K documents was suddenly asked to support 500K. Action: immediately assessed impact on latency and cost, proposed tiered retrieval as a scope-preserving alternative, renegotiated timeline with stakeholder. Result: delivered within revised timeline with no quality regression.

Q7 [EY Behavioral]

How do you stay updated in this field given how fast AI moves?

- Weekly: Arxiv Sanity / Papers With Code — scan new RAG and LLM papers
- Practitioner sources: LangChain blog, Azure OpenAI release notes, Hugging Face blog
- Build-to-learn: every new concept gets a small prototype before it goes into production consideration
- Community: LangGraph Discord, AI Engineer community, internal Accenture AI CoP
- Filter signal from noise: only adopt tools that solve a real problem you currently have

Q8 [EY Behavioral]

Describe a technical disagreement with a teammate and how you resolved it.

Framework: State the disagreement clearly, show you understood their position, explain how you reached resolution (data/proof, not authority).

Example: Disagreed on whether to use semantic chunking vs. recursive character splitting. Teammate preferred semantic for quality. I preferred recursive for latency. Resolution: benchmarked both on our actual document set — recursive was 3x faster with <5% quality difference. Chose recursive for production, flagged semantic for future upgrade when we have offline pipeline capacity.

Q9 [EY — Close Every Answer With This]

Explain how your evaluation and monitoring approach ensures the fix is actually working.

This is the pattern EY expects at the end of EVERY scenario answer:

- Define the metric that proves success (e.g. faithfulness score, p95 latency, cost per query)
- Set a baseline before the fix is deployed
- Measure the same metric after deployment
- Set a monitoring alert if the metric regresses beyond a threshold
- Schedule a periodic evaluation run (weekly or monthly) on a golden test set

Example close: 'And we would know the fix is working if faithfulness score returns above 0.85 on our 50-query golden test set, measured weekly via our RAGAS evaluation pipeline.'

■ EY Winner Pattern

'Always mention evaluation and monitoring, even if not directly asked. In every scenario, I tried to close by explaining how we would know if the fix is actually working. That landed well every time.' — EY Candidate

SECTION 4 — EY vs Infosys | Interview Pattern Analysis

Dimension	Infosys	EY
Question style	'What is X?'	'X is broken in prod. Fix it.'
Test focus	Conceptual knowledge	Architectural judgment + client translation
Role signal	ML Engineer / Builder	Consulting GenAI Engineer
What they hire for	Can you build it?	Can you own it AND explain it to a client?
Live coding	Possible	None — whiteboard reasoning only
Evaluation asked	Rarely	Expected in every answer
Access control	Not asked	Major theme
Cost awareness	Rarely	Core scenario

The 3 themes EY tested across ALL rounds:

1. **Grounding** — retrieval-time filtering is always better than generation-time filtering. Fallback > hallucination. Never trust the LLM to self-censor.
2. **Cost control** — model routing by task complexity, caching, prompt sizing, duplicate call detection.
3. **Access control** — metadata filtering at retrieval. Wrong chunks must never reach the prompt. Access control leakage is a compliance bug.

SECTION 5 — Quick-Reference Cheat Sheet

Answer Structure for Every EY Scenario

Diagnose → Decide → Fix → Verify

Never fix before diagnosing. Always close with how you'd verify the fix worked.

Key Numbers to Know

Concept	Rule of Thumb
Chunk size (prose)	256–512 tokens
Chunk overlap	10–20% of chunk size
Similarity threshold	Discard chunks below 0.75 cosine similarity
Top-k retrieval	Retrieve 10–20, re-rank, pass top 3–5 to LLM
Tokens per word	~1.3 tokens/word (English). Use tiktoken to verify.
Faithfulness target	>0.85 for production RAG systems
Cost alert thresholds	Alert at 50% and 80% of monthly budget

The Sentence That Wins the Client Round

"No AI system is 100% accurate — just like humans doing the same work make mistakes. The real question is: how do we detect and contain mistakes?"

Then point to: guardrails, groundedness checks, fallback responses, human-in-the-loop for high-stakes queries.

Dense vs Sparse — One-Line Summary

Dense misses exact codes and acronyms. Sparse misses semantic intent. Hybrid retrieval + re-ranking covers both blind spots.

Access Control Rule (Never Forget)

Access control must happen AT retrieval time, not as a post-filter. Wrong chunks must never reach the prompt. Never trust the LLM to withhold information.

SECTION 6 — Square Yards Interview | ML · Python · DSA · HR

Focus: project deep-dive + core ML concepts + Python internals + DSA problem + HR. Projects led every question — interviewers drilled into why and how, not just what.

Part A — Project Discussion & Multi-Agent Architecture

Q6.1 [Square Yards]

Walk me through your project — problem, workflow, architecture, how LLMs work inside it.

Structure your answer in 4 layers:

1. **Problem** — What business problem does it solve? What was the pain before? (1–2 sentences, non-technical)
2. **Workflow** — End-to-end user journey: input → processing → output. Draw it mentally before speaking.
3. **Architecture** — Key components: document ingestion, embedding, vector DB, retrieval, LLM, output layer. Name each and why it's there.
4. **LLM role** — LLM is NOT the whole system. It is one component: it takes a grounded prompt (system instructions + retrieved context + user query) and generates a structured response.

Closing line interviewers love: 'The LLM handles language understanding and generation. Every other component — chunking, retrieval, re-ranking — exists to give the LLM the right context so it doesn't have to guess.'

Q6.2 [Square Yards]

How do you assign tasks to different AI agents in a multi-agent system?

Three common patterns for task assignment in multi-agent systems:

1. Router / Orchestrator Agent

A central agent receives the user query, classifies intent, and routes to the specialist agent best suited for that task. The router uses an LLM call or a classifier to decide.

Example: user query → router identifies it as a 'document retrieval' task → sends to RAG agent, not the summarisation agent.

2. Conditional Edges in LangGraph

Each node in the graph represents an agent. Edges are conditional — based on the state value returned by a node, the graph decides which agent (node) runs next.

Example: retrieval node returns `retrieved_docs=[]` → edge routes to `fallback_agent`. `Retrieved_docs` non-empty → routes to `generation_agent`.

3. Supervisor + Worker Pattern

A supervisor LLM receives the task, breaks it into subtasks, and delegates each subtask to a specialised worker agent. Workers report back; supervisor synthesises the final answer.

Useful for complex multi-step tasks: research agent + calculation agent + formatting agent all coordinated by one supervisor.

Key principle: Each agent should have a single, well-defined responsibility (SRP). Agents communicate through shared state, not direct function calls — this makes the system observable and debuggable.

■ Future Scope Frame

When asked about future scope — always mention: adding more specialised agents, improving evaluation/observability, and moving from reactive (query-response) to proactive (agent-initiated) workflows.

Part B — Machine Learning Concepts

Q6.3 [Square Yards]

Supervised vs Unsupervised Learning — difference and examples.

	Supervised Learning	Unsupervised Learning
Definition	Model learns from labelled data (input + correct output pairs)	Model finds patterns in unlabelled data (input only)
Goal	Predict a known target variable	Discover hidden structure or groupings
Examples	Classification (spam/not spam), Regression (house price prediction)	Clustering (customer segments), Dimensionality reduction (PCA)
Algorithms	Linear Regression, Decision Tree, SVM, KNN, Neural Networks	K-Means, DBSCAN, Autoencoders, PCA, t-SNE
When to use	You have labelled training data and a clear target	You have no labels and want to explore data structure

Semi-supervised: small labelled set + large unlabelled set. Common in real-world ML where labelling is expensive. **Self-supervised:** model generates its own labels from data (how LLMs are pre-trained — predict next token).

Q6.4 [Square Yards]**What is KNN? How does it work?**

K-Nearest Neighbours (KNN) is a simple, non-parametric algorithm. It makes predictions based on the k most similar training examples to the query point.

How it works (classification):

1. Receive a new data point to classify
2. Calculate distance from that point to every training point (Euclidean or cosine)
3. Select the k nearest neighbours
4. Take a majority vote — the most common class among k neighbours is the prediction

Aspect	Detail
Lazy learner	No training phase — all computation happens at prediction time
k choice	Small k = complex boundary (overfit); Large k = smooth boundary (underfit). Use cross-validation.
Curse of dimensionality	KNN degrades in high-dimensional spaces — distances become meaningless
Scaling required	Always normalise features first — otherwise high-range features dominate distance
Time complexity	O(n) per prediction — slow on large datasets

Connection to RAG: Vector similarity search in a vector DB is conceptually KNN — find the k nearest embedding vectors to the query embedding. That's why understanding KNN is directly relevant to your project.

■ Bridge to Your Project

The KNN → vector search connection is a powerful answer when interviewer asks 'how does KNN relate to your GenAI work?' — dense retrieval IS approximate KNN in embedding space.

Q6.5 [Square Yards]**What is Euclidean Distance?**

Euclidean Distance is the straight-line distance between two points in n-dimensional space — the geometric distance most of us learned in school.

Formula (2D): $d = \sqrt{(x_2 - x_1)^2 + (y_2 - y_1)^2}$

General (n dimensions): $d = \sqrt{\sum (x_i - y_i)^2}$

Properties:

- Sensitive to the magnitude of values — features on different scales must be normalised first
- Works well in low-dimensional spaces with continuous features
- Degrades in very high dimensions (all points become equidistant — curse of dimensionality)

Q6.6 [Square Yards]

Cosine Similarity vs Euclidean Distance — when to use which?

	Cosine Similarity	Euclidean Distance
Measures	Angle between two vectors (direction)	Straight-line distance (magnitude)
Formula	$\cos(\theta) = (A \cdot B) / (A \times B)$	$d = \sqrt{\sum (x_i - y_i)^2}$
Range	-1 to 1 (1 = identical direction)	0 to ∞ (0 = same point)
Scale sensitivity	Not sensitive — normalises by magnitude	Sensitive — large values dominate
Best for	Text embeddings, document similarity, sparse high-dim vectors	Image pixels, tabular data, low-dim geometric problems
In RAG	Default for embedding similarity search	Used in image retrieval or spatial data

Key insight: For text, two documents with the same words but different lengths will have very different Euclidean distances but similar cosine similarity. That's why cosine is preferred for NLP — we care about direction (meaning), not magnitude (length).

Part C — Python Internals

Q6.7 [Square Yards]

What are Python Decorators?

A **decorator** is a function that wraps another function to extend or modify its behaviour without changing its source code. It follows the **Open/Closed Principle** — open for extension, closed for modification.

Basic structure:

```
def my_decorator(func):
    def wrapper(*args, **kwargs):
        print('Before call')
        result = func(*args, **kwargs)
        print('After call')
        return result
    return wrapper

@my_decorator
def greet(name):
    print(f'Hello {name}')
```

Real-world uses:

- **@retry** — retry a function on failure (tenacity library — used in LLM calls)
- **@lru_cache** — memoize expensive function results
- **@app.route** — FastAPI/Flask route registration
- **@property** — make a method behave like an attribute
- **@staticmethod / @classmethod** — control method binding

■ Project Connection

Connect to your project: '@retry from tenacity is a decorator I use to wrap Azure OpenAI calls — it handles transient 429 rate limit errors with exponential backoff automatically.'

Q6.8 [Square Yards]

What are Python Generators?

A **generator** is a function that uses **yield** instead of **return**. It produces values one at a time and pauses execution between yields — it does not compute all values upfront. This makes it memory-efficient for large datasets.

Regular function vs Generator:

```
# Regular — loads all 1M numbers into memory at once
def get_numbers(): return [i for i in range(1_000_000)]

# Generator — produces one number at a time, uses ~O(1) memory
def get_numbers():
    for i in range(1_000_000):
        yield i
```

Key properties:

- Returns a generator object (lazy iterator) — values computed only when requested
- Can only be iterated once — not reusable like a list
- Generator expressions: `(x*2 for x in range(10))` — like list comp but lazy

Practical use in AI/ML: streaming large document chunks through a pipeline without loading all into memory. LangChain's document loaders use generators internally for this reason.

Q6.9 [Square Yards]

Difference between List, Set, Tuple, and Dictionary?

	List	Set	Tuple	Dictionary
Syntax	[]	{ }	()	{ k: v }
Ordered	Yes (indexed)	No	Yes (indexed)	Yes (Python 3.7+)
Mutable	Yes	Yes	No	Yes (keys immutable)
Duplicates	Allowed	Not allowed	Allowed	Keys unique, values allowed
Lookup speed	O(n)	O(1) avg	O(n)	O(1) avg
Use case	Ordered collection, append/remove	Unique items, membership test	Fixed data, dict key	Key-value mapping
Example	[1,2,2,3]	{1,2,3}	(lat, lon)	{'name': 'Raj'}

Immutability rule: Tuples can be used as dictionary keys (immutable, hashable). Lists cannot.

Set for deduplication: Converting a list to a set is the fastest way to remove duplicates — O(n).

Part D — DSA Problem

Q6.10 [Square Yards DSA]

Given a list of strings, find the string with the maximum number of unique characters.

Example: ['apple','banana','orange','aabbcc','abcdef'] → **Output:** 'abcdef'

Approach — Think out loud first:

For each string, count its unique characters. The string with the highest unique count is the answer.
A set gives us unique characters in O(k) where k = string length.

Solution 1 — One-liner (most Pythonic):

```
words = ['apple', 'banana', 'orange', 'aabbcc', 'abcdef']
result = max(words, key=lambda w: len(set(w)))
print(result) # abcdef
```

Solution 2 — Explicit loop (easier to explain in interview):


```
def max_unique_chars(words):
    best_word = ''
    best_count = 0
    for word in words:
        unique = len(set(word))
        if unique > best_count:
            best_count = unique
            best_word = word
    return best_word

print(max_unique_chars(['apple', 'banana', 'orange', 'aabbcc', 'abcdef']))
# Output: abcdef
```

Aspect	Detail
Time complexity	$O(n \times k)$ where n = number of strings, k = avg string length
Space complexity	$O(k)$ for the set of unique chars per word
Edge cases to mention	Empty list → return "" or raise. Tie (two strings with same unique count) → returns first found.
set(word)	Converts string to set of unique characters — 'apple' → {'a','p','l','e'} (4 unique)

■ DSA Interview Protocol

In any DSA interview: (1) clarify edge cases first, (2) state your approach before coding, (3) state time/space complexity at the end. Never jump straight to code.

Part E — HR Questions

Q6.11 [Square Yards HR]

Why did you choose AI/ML as a career?

Structure: Curiosity → Exposure → Conviction → Direction

Template answer:

'I was drawn to AI/ML because it sits at the intersection of problem-solving and real impact. My first exposure was [specific project/moment]. What convinced me was realising that AI doesn't just automate tasks — it enables solutions that weren't possible before. Now at Accenture, working on LLM-based diagnostics, I see that directly. I chose this field because the learning curve never flattens — every project pushes me into new territory.'

What to avoid: 'It's the future' or 'it pays well' — both signal surface-level thinking. Ground it in a specific moment or project.

Q6.12 [Square Yards HR]**What are your career goals?**

Frame: Short-term (1–2 years) → Mid-term (3–5 years) → Long-term vision

Template answer:

'Short-term, I want to deepen my expertise in production GenAI systems — specifically around evaluation, reliability, and multi-agent architectures. Mid-term, I see myself leading end-to-end AI solution delivery for enterprise clients, bridging technical implementation with business outcomes. Long-term, I want to be someone who helps define how organisations adopt AI responsibly — not just build it, but govern it well.'

Key: Show ambition but ground it in current role. Don't say 'I want to become a data scientist' if you're already in AI/ML.

Q6.13 [Square Yards HR]**What are your strengths and weaknesses?****Strength — choose one that's verifiable from your work:**

'My strength is systems thinking — I naturally look at how components fit together before implementing any one piece. In my LLM project, this meant I designed the retrieval and evaluation pipeline together from day one, which prevented costly rework later.'

Weakness — the formula: real weakness + active mitigation + progress:

'I sometimes go deep into optimising something before confirming it's the right thing to optimise — I over-engineer early. I've been working on this by timeboxing exploration phases and validating assumptions with stakeholders before diving into implementation. It has improved my delivery speed.'

What to avoid for weakness: 'I work too hard' or 'I'm a perfectionist' — interviewers see through these immediately. A real, contained weakness shows self-awareness.

Square Yards Key Takeaway

'Projects can lead to much deeper technical discussions than expected. Knowing how and why every component works is just as important as building it.'

The KNN → vector search bridge, the decorator → retry pattern connection, the DSA protocol — all of these are about depth of understanding, not breadth of memorisation.

SECTION 7 — FAANG-Level AI Engineer Cheat Sheet | From RAG to Agents to Security

Source: AI Engineer Interview Cheat Sheet (Syamala Jayanth). Covers all 15 topic areas — FAANG-level and beyond. It's not just about knowing AI — it's about building, debugging, and operating AI systems in the real world.

7.1 — LLM Fundamentals

Concept	What to Know
Transformers (encoder/decoder)	Encoder: bidirectional attention (BERT). Decoder: causal/autoregressive (GPT). Encoder-decoder: seq2seq tasks (T5, BART).
Attention (Q, K, V, multi-head)	$Q=$ query, $K=$ key, $V=$ value. Score = $\text{softmax}(QK^T/d) \cdot V$. Multi-head runs H attention heads in parallel — each learns different relationships.
Tokenization (BPE, WordPiece)	BPE merges frequent byte pairs iteratively. WordPiece maximises likelihood of training data. ~1.3 tokens/word in English. Use tiktoken to count.
Positional encoding	Transformers have no inherent order sense. Sinusoidal (original) or learned (GPT) or RoPE (LLaMA) positional vectors added to embeddings.
KV cache	Stores computed Key/Value pairs from previous tokens so they aren't recomputed each step. Critical for inference speed. Memory bottleneck at long contexts.
Context windows	Max tokens the model can attend to in one call. GPT-4o: 128k. Claude: 200k. Larger = slower + costlier. Relevant content must be in-window.
Sampling (temperature, top-k, top-p)	Temperature scales logits before softmax. top-k hard-limits vocabulary. top-p dynamically cuts vocabulary at cumulative probability. Use all three together.
Mixture of Experts (MoE)	Model has N expert FFN layers; a router activates only k experts per token. Scales parameter count without scaling compute proportionally (GPT-4, Mixtral).
Inference (greedy vs sampling)	Greedy: always pick highest probability token. Deterministic but repetitive. Sampling: draw from distribution. Beam search: keep top-B sequences at each step.

7.2 — RAG (Advanced Topics Beyond Basics)

Advanced Concept	What to Know
Hybrid search (lexical + semantic)	BM25 for exact keyword match + dense vectors for semantic. Merge results with Reciprocal Rank Fusion (RRF). Covers each other's blind spots.
Reranking	After retrieval, a cross-encoder re-scores each (query, chunk) pair with full attention. Slower but more accurate than cosine similarity. Use Cohere Rerank or BGE.
Query rewriting	Rewrite the user query before retrieval to improve recall. Techniques: HyDE, step-back prompting, multi-query generation, query expansion.
HyDE (Hypothetical Document Embedding)	Generate a hypothetical answer to the query → embed that answer → use its embedding for retrieval. Bridges the query-document vocabulary gap.
Multi-hop retrieval	Answer requires combining information from multiple chunks. Iterative: retrieve → reason → retrieve again. Used in complex Q&A; over large corpora.
GraphRAG	Store knowledge as a graph (entities + relationships). Retrieve subgraphs relevant to query. Better for relational / multi-entity queries than flat vector search.
Agentic RAG	RAG inside an agent loop — agent decides when to retrieve, what to query, whether to retrieve again based on initial results. More powerful, harder to debug.
RAG Evaluation (recall@k, faithfulness)	Recall@k: how much ground truth is in top-k retrieved chunks. Faithfulness: does the answer only use retrieved context? Use RAGAS for both.

7.3 — AI Agents

Concept	What to Know
Tool calling	LLM outputs a structured function call (name + args). Runtime executes the tool and returns result to LLM. Foundation of all agentic behaviour.
Planning & reasoning	ReAct (Reason + Act): interleave reasoning steps with tool calls. Chain-of-thought before acting. Tree-of-thoughts for branching decisions.
State management	State = typed dict flowing through all nodes. Each node reads + writes state. LangGraph checkpoint saves state after each node for recovery.
Memory (short/long-term)	Short: conversation history in context window. Long: vector DB of past interactions, retrieved when relevant. External: databases, files.
Workflows vs agents	Workflow: deterministic sequence of steps (hardcoded). Agent: LLM decides next step dynamically. Agents are powerful but less predictable.
MCP (Model Context Protocol)	Anthropic's open standard for connecting LLMs to external tools/data sources. Replaces bespoke tool integrations with a uniform protocol.
Human-in-the-loop	Agent pauses at defined checkpoints and waits for human approval before continuing. Critical for high-stakes actions (deleting data, sending emails).
Error handling & retries	Node-level try/except. Exponential backoff with tenacity. Conditional edges to fallback nodes. Dead-letter state after N failures.
Authorization & permissions	Agent should only have access to tools/data the user is authorised for. Scope tool permissions at the session level, not globally.
Observability & tracing	Log every tool call + input + output + latency. Use LangSmith, LangFuse, or OpenTelemetry. Tracing is non-negotiable in multi-agent systems.

7.4 — Model Adaptation

Concept	What to Know
Prompt engineering	Zero-shot, few-shot, chain-of-thought, structured output prompting. System prompt defines role and constraints. Output parsers enforce schema.
RAG vs fine-tuning	RAG: external knowledge, updatable, no training cost. Fine-tuning: bakes knowledge into weights, better for style/format adaptation. Use RAG first.
DPO (Direct Preference Optimisation)	Trains model on (preferred, rejected) response pairs without a separate reward model. More stable than RLHF. Used for alignment and instruction following.
Catastrophic forgetting	Fine-tuning on new data causes model to forget old knowledge. Mitigations: LoRA (low-rank adapters), EWC, mixed training with original data.
Dataset quality	Garbage in, garbage out. Deduplication, diversity, quality filtering are more important than dataset size. Curated 10K examples >> noisy 1M examples.
Instruction tuning	Fine-tune on (instruction, response) pairs to make base model follow natural language instructions. Foundation of chat models (FLAN, Alpaca, etc.).

7.5 — Evaluation

Concept	What to Know
Golden datasets	Curated (query, expected answer, relevant chunks) triples. Ground truth for offline evaluation. Must be maintained as the system evolves.
Offline evaluation	Evaluate on golden dataset before deployment. Fast, cheap, controlled. Limitation: doesn't capture real user query distribution.
Online evaluation	A/B testing in production. Capture implicit signals (thumbs up/down, session length, follow-up queries). Ground truth for future golden datasets.
LLM-as-a-judge	Use a capable LLM (GPT-4o) to score another LLM's outputs on a rubric: accuracy, groundedness, relevance, completeness. Scale > human eval.
Retrieval metrics	Recall@K: fraction of relevant docs in top-K. MRR: mean reciprocal rank of first relevant result. nDCG: discounted cumulative gain (ranking quality).
Generation metrics	Faithfulness: answer grounded in context only. Relevance: on-topic for the question. Completeness: all aspects of question addressed.
Hallucination evaluation	NLI-based: does context entail the answer? LLM-as-a-judge: is each claim supported by retrieved context? FActScore for fact-level granularity.
Regression testing	Every deployment must pass evaluation gate: metric scores must not drop below baseline. Automate in CI/CD pipeline.

7.6 — Production AI

Concept	What to Know
Latency optimisation	KV cache, speculative decoding, quantisation (INT8/INT4), smaller models for simple tasks, streaming (first-token latency vs full response).
Cost optimisation	Model routing by complexity. Prompt compression. Semantic caching (GPTCache). Batching async requests. Avoid sending full docs when chunks suffice.
Caching	Exact match: cache identical prompts. Semantic cache: cache prompts within cosine similarity threshold. Reduces redundant LLM calls by 30–70%.
Batching	Group multiple requests into one LLM call. Reduces per-request overhead. Use with async queue. Trade-off: adds latency for individual requests.
Streaming	Stream tokens as they are generated (SSE). Dramatically improves perceived latency. Requires streaming-compatible output parsers.
Model routing	Route simple queries to cheaper/faster models (GPT-3.5, Haiku). Reserve frontier models (GPT-4o, Opus) for complex reasoning. Saves 60–80% cost.
Rate limiting	Hard cap per API key per minute. Client-side: exponential backoff + jitter. Multiple API keys for load distribution. Queue requests during spikes.
Concurrency & autoscaling	Async LLM calls with <code>asyncio.gather</code> + Semaphore. Horizontal scaling of inference endpoints. Kubernetes HPA on token throughput metric.
Observability & monitoring	Instrument every LLM call: latency, token count, cost, model version, response. Alert on p95 latency, error rate, cost per query.
Tracing (logs, metrics, traces)	Distributed tracing across the full agent graph. LangSmith, LangFuse, or OpenTelemetry. Correlate user query → retrieval → LLM call → response in one trace.

7.7 — Security & Safety

Threat / Concept	What to Know
Prompt injection	Malicious content in retrieved docs or user input overrides system instructions. Mitigations: input sanitisation, instruction hierarchy, separate system prompt from user data.
Data leakage	Model inadvertently reveals training data or other users' data in responses. Mitigations: access control at retrieval, output filtering, PII detection before response.
PII handling	Detect and redact PII (names, emails, SSNs) before indexing and before sending to LLM APIs. Use Microsoft Presidio or AWS Comprehend for detection.
Tool abuse	Agent misuses a tool (deletes data, sends emails without authorisation). Mitigations: principle of least privilege, human-in-the-loop for destructive actions, tool sandboxing.
Guardrails	Input and output validation layer. Check for off-topic queries, harmful content, PII, and policy violations. Guardrails AI, NVIDIA NeMo Guardrails.
Tenant isolation	In multi-tenant systems, ensure one tenant's data cannot appear in another's responses. Metadata filtering at retrieval + namespace isolation in vector DB.
Multi-tenancy	Design: per-tenant namespaces in vector DB, per-tenant API keys with rate limits, per-tenant audit logs. Never share index or context across tenants.
Output validation	Validate LLM output before returning to user: schema check, hallucination check, PII scan, policy filter. Reject and fallback if validation fails.
Safety policies	Define and enforce: what topics the system can/cannot discuss. Implement as system prompt constraints + output classifier. Log all policy violations.

■ Security Interview Pattern

Security question pattern: 'How do you prevent prompt injection in an agentic RAG system?' Answer: treat every retrieved chunk as untrusted user input. Never allow retrieved content to override system instructions.

7.8 — Classical ML Fundamentals

Concept	What to Know
Classification & regression	Classification: predict discrete class. Regression: predict continuous value. Same algorithms, different output layer and loss function.
Bias vs variance	Bias: model too simple, misses patterns (underfitting). Variance: model too complex, memorises noise (overfitting). Optimal = low bias + low variance.
Overfitting & regularisation	Overfitting: perfect on train, poor on test. Regularisation: L1 (Lasso, sparse weights), L2 (Ridge, small weights), Dropout (neural nets).
Precision, Recall, F1	Precision = $TP/(TP+FP)$. Recall = $TP/(TP+FN)$. $F1 = 2 \cdot P \cdot R / (P+R)$. Use F1 when class imbalance exists. Choose precision or recall based on cost of each error type.
ROC-AUC	ROC plots TPR vs FPR at all thresholds. AUC = probability that model ranks a random positive higher than a random negative. AUC=0.5 is random; 1.0 is perfect.
Feature engineering	Create informative input features from raw data. Normalisation, one-hot encoding, embeddings, interaction terms. Often more impactful than model choice.
Data leakage	Training data contains information that wouldn't be available at prediction time. Causes falsely high eval metrics. Always split before any preprocessing.
Cross-validation	K-fold: split data into K folds, train on K-1, test on 1, rotate. Stratified for class imbalance. More reliable metric estimate than single train/test split.
Model drift	Model performance degrades over time as real-world data distribution shifts. Monitor prediction distribution, feature distribution, and accuracy on labelled samples.
Class imbalance	Minority class underrepresented. Fixes: oversampling (SMOTE), undersampling, class weights, threshold tuning. Always use F1/AUC not accuracy for evaluation.

7.9 — Engineering Fundamentals

Area	Key Topics to Know
Python (OOP, async, decorators)	Classes, inheritance, dunder methods. asyncio, await, gather, Semaphore. Decorators (@retry, @lru_cache). Generators (yield). Type hints.
DSA (arrays, strings, trees, graphs, DP)	Arrays/strings: two-pointer, sliding window, hash map. Trees: DFS/BFS, binary search tree. Graphs: BFS/DFS, topological sort. DP: memoisation, tabulation.
SQL (joins, GROUP BY, window functions)	INNER/LEFT/RIGHT JOIN. GROUP BY + HAVING. Window: ROW_NUMBER(), RANK(), LAG(), LEAD(), SUM() OVER(PARTITION BY). CTEs and subqueries
APIs (REST, GraphQL)	REST: stateless, HTTP verbs (GET/POST/PUT/DELETE), status codes. GraphQL: single endpoint, query exactly what you need, reduces over/under-fetching.
Kubernetes	Pod: smallest deployable unit. Deployment: manages pod replicas. Service: exposes pods. HPA: horizontal pod autoscaler. ConfigMap/Secret for config.
CI/CD	Continuous Integration: auto-test on every commit. Continuous Deployment: auto-deploy on passing tests. Evaluation gate in AI CI/CD: metric score must pass.
Cloud (AWS/GCP/Azure)	Compute: EC2/GCE/VMs. Serverless: Lambda/Cloud Functions/Azure Functions. Storage: S3/GCS/Blob. LLM APIs: Bedrock/Vertex/Azure OpenAI.
Databases (SQL/NoSQL)	SQL: ACID, joins, structured schema. NoSQL: eventual consistency, flexible schema. Vector DB: approximate nearest neighbour search. Choose by query pattern.
Queues (Kafka, RabbitMQ)	Kafka: distributed, high-throughput, durable, replay. RabbitMQ: traditional message broker, ACK/NACK, routing. Use for async LLM request batching.

7.10 — System Design for AI

Common system design scenarios and how to approach them:

Design Scenario	Key Components to Address
RAG platform	Ingestion pipeline → chunking → embedding → vector DB → hybrid retrieval → reranking → LLM → evaluation → monitoring
AI assistant / chatbot	Session management → history truncation → RAG retrieval → response streaming → guardrails → feedback collection
Agent platform	Tool registry → state management → orchestrator → worker agents → checkpointing → human-in-the-loop → tracing
LLM serving	Model endpoints → load balancing → KV cache → batching → autoscaling → fallback models → cost tracking
High-scale inference	Request queue (Kafka) → async workers → model pool → semantic cache → rate limiting → circuit breaker → observability
Multi-tenant AI system	Tenant namespacing → access control at retrieval → isolated rate limits → per-tenant audit logs → data isolation

Universal system design framework for AI: Start with data flow → add retrieval layer → add generation layer → add evaluation → add cost controls → add security → add observability. Always discuss trade-offs and cost.

7.11 — AI Security Deep Dive

Security Concept	Deep Dive
Prompt injection attacks	Direct: user inserts malicious instruction in their query. Indirect: malicious content embedded in retrieved document overrides system prompt. Defence: treat all external content as data, not instructions. Use delimiters.
Data isolation	Namespace isolation in vector DB (Pinecone namespaces, pgvector schema-per-tenant). Never share embedding index across tenants. Validate tenant ID on every retrieval call.
Sensitive data handling	Detect PII before indexing (Presidio). Encrypt at rest (AES-256). Encrypt in transit (TLS 1.3). Use token-level masking for LLM API calls. Audit all data access.
Tool / function abuse	Agents can be manipulated into calling destructive tools. Defence: least privilege (read-only tools by default), confirmation step for write/delete, sandboxed tool execution environment.
Model safeguards	Input classifiers (off-topic, harmful content). Output classifiers (PII, policy violations). Jailbreak detection. Confidence thresholds. Fail-safe defaults.
Red-teaming	Adversarial testing: try to jailbreak, inject, leak data, abuse tools. Build a red-team test suite. Run before every major deployment. Document findings.
Audit logs	Log every request: user ID, timestamp, input, retrieved chunks, output, model version, latency, cost. Tamper-evident. Retention policy aligned with compliance requirements.
Compliance (GDPR, etc.)	GDPR: right to erasure (delete user's chunks from vector DB). Data residency. Consent for data processing. Azure EU data boundary for EU clients.
Secure embeddings & storage	Embeddings can partially reconstruct original text. Treat as sensitive. Store in access-controlled vector DB. Encrypt embedding files. Don't log raw embeddings.

7.12 — Observability & Debugging

What to Instrument	Why It Matters
Tracing (request → prompt → model output)	Full request trace from user input to final response. Shows where latency lives and where errors originate. Use trace IDs to correlate across services.
Latency breakdown	Measure separately: retrieval latency, reranking latency, LLM first-token latency, full response latency. Optimise the biggest bottleneck first.
Token usage	Input tokens + output tokens per request. Track daily/weekly trends. Token count spikes = prompt bloat bug. Token count drops = truncation bug.
Model / version tracking	Log which model + version served each request. Essential for debugging performance regressions after model updates or API changes.
Retrieval scores & top-K docs	Log similarity scores and which chunks were retrieved for each query. Low scores = embedding or index problem. Irrelevant chunks = metadata filter problem.
Tool call logs	For agents: log every tool name, input args, output, and latency. Tool failures and unexpected outputs are the most common agentic failure mode.
Evaluation scores	Log RAGAS scores (or equivalent) periodically. Alert if faithfulness drops below threshold. Trend analysis reveals gradual quality degradation.
Cost monitoring	Per-request cost = (input_tokens × input_price + output_tokens × output_price). Alert at 50%/80% of budget. Dashboard by model, endpoint, user segment.

7.13 — MLOps / Deployment

Concept	What to Know
Prompt versioning	Version control prompts like code. Tag each prompt version. Track which version served each request. Roll back on performance regression.
Evaluation gate	In CI/CD pipeline: new deployment must pass automated eval (RAGAS scores above baseline) before going to production. Blocks bad deployments automatically.
Canary deployment	Deploy new version to 5–10% of traffic. Monitor metrics vs control group. Promote to 100% if metrics hold; rollback if they degrade. Reduces blast radius.
Model monitoring	Track prediction distribution, confidence score distribution, retrieval score trends. Alert on distribution shift — early warning of degrading quality.
Data / concept drift	Data drift: input feature distribution changes. Concept drift: relationship between inputs and correct outputs changes. Monitor both separately.
GPU inference	Batch requests to maximise GPU utilisation. Use quantisation (INT8/INT4) for 2–4x speedup with minimal quality loss. vLLM for PagedAttention (KV cache efficiency).
Autoscaling	Scale inference endpoints on token throughput or request queue depth, not just CPU. Kubernetes HPA with custom metrics. Pre-warm instances for latency SLAs.

7.14 — Interview Coding Patterns

Category	Patterns to Master
Python internals	generators (yield), decorators (wrapper), iterators (<code>__iter__</code> / <code>__next__</code>), OOP (inheritance, dunder), error handling (try/except/finally), multiprocessing vs asyncio
Arrays & strings	Two-pointer, sliding window, prefix sum, hash map for O(1) lookup, anagram check (Counter), unique char count (set)
Stacks & queues	Monotonic stack (next greater element), queue for BFS, deque for sliding window max, priority queue (heapq) for top-K
Trees & graphs	DFS (recursive/iterative), BFS (level-order), binary search, topological sort (Kahn's), union-find for connectivity
Binary search	Search sorted array, find boundary condition, search answer space (not just array). Template: lo, hi, mid = (lo+hi)//2
Sliding window	Fixed size: move window right. Variable size: shrink from left when constraint violated. Track window state with hash map.
Dynamic programming	Identify: overlapping subproblems + optimal substructure. Memoisation (top-down) vs tabulation (bottom-up). State = what changes between subproblems.
SQL patterns	GROUP BY + HAVING for aggregation filters. ROW_NUMBER() OVER(PARTITION BY x ORDER BY y) for ranking per group. LAG/LEAD for period-over-period comparisons.

7.15 — Behavioural Rounds

Question Type	What They're Testing + Answer Frame
Deep dive into AI projects	Can you explain every architectural decision? Use: Problem → Architecture → Why each component → What you'd do differently now.
Production failure story	Do you own failures or deflect? Use: What broke → How you diagnosed → What you did → What you changed to prevent recurrence.
Technical disagreement	Can you collaborate without capitulating? Use: State their view fairly → State your view → How you resolved (data/proof, not authority) → Outcome.
Trade-off decisions	Do you think in systems? Use: Options considered → Criteria used → Decision made → Result → What you'd revisit with new information.
Leadership & ownership	Do you take initiative? Use: Situation where no one owned the problem → What you did → Impact → What it cost you and why it was worth it.
Handling ambiguity	Can you move without perfect information? Use: Unclear situation → Assumptions you made explicit → How you validated them → How you adjusted.
Cross-functional collaboration	Can you work with non-engineers? Use: Specific interaction with product/design/client → Translation challenge → How you bridged it → Outcome.
Impact & results	Can you quantify your work? Always have: before metric, after metric, and what you specifically did that caused the change. Vague impact = weak answer.
Situational questions	What would you do if...? Use the same STAR frame but hypothetically. Show your decision process, not just the decision.

System Design Quick-Fire Q&A;

Scenario Question	Answer Framework
How to debug wrong answers in RAG?	1. Check retrieval scores — are right chunks being retrieved? 2. Check faithfulness — is LLM ignoring context? 3. Check prompt — is grounding instruction clear? 4. Add LLM-as-a-judge step.
How to handle 1M queries/day with low latency & no hallucinations?	Semantic cache (hit rate 30–70%). Async request queue. Model routing (small model for simple). Streaming. Groundedness check before response. HPA autoscaling.
How to evaluate a RAG pipeline?	RAGAS: Context Recall + Context Precision + Faithfulness + Answer Relevancy. Golden dataset. LLM-as-a-judge. Online A/B eval. Regression gate in CI/CD.
What to do when retrieval is poor?	Check embedding model — is it domain-appropriate? Add BM25 hybrid retrieval. Tune chunk size/overlap. Add reranker. Try query rewriting or HyDE. Inspect metadata filters.
How to handle tool failures in agents?	Node-level try/except → error state. Conditional edge to fallback node. Tenacity retry with exponential backoff. Dead-letter after N failures → human review.
How to design for multi-tenancy?	Namespace isolation in vector DB. Per-tenant API keys + rate limits. Access control at retrieval (metadata filter = tenant_id). Per-tenant audit logs. Never share context across tenants.

■ FAANG-Level Mindset

The cheat sheet's closing truth: 'It's not just about knowing AI — it's about building, debugging, and operating AI systems in the real world.' Every interview answer should demonstrate operational thinking: build it → test it → monitor it → improve it.

SECTION 8 — AI/ML Interview Experience | ML Models · OCR · RAG · LangGraph

3-round interview: project-driven throughout. Discussion arc: What did you build? → Why that technology? → How did you measure the result? This section fills the gaps not covered in earlier sections: XGBoost, Random Forest, Decision Trees, and OCR.

Part A — RAG Follow-Up Questions (New Angles)

Q8.1 [AI/ML Interview]

What are the limitations of RAG?

Limitation	Root Cause	Mitigation
Retrieval failure	Wrong chunks retrieved — embedding model doesn't capture domain semantics	Hybrid search (BM25 + dense), domain-specific embeddings, query rewriting
Context window overflow	Too many retrieved chunks exceed LLM token limit	Reranking — keep top-3 to 5 only; context compression
Chunking boundary problem	Key information split across chunks	Chunk overlap, semantic chunking, parent-child retrieval
Hallucination despite retrieval	LLM ignores retrieved context and generates from parametric memory	Stronger grounding instructions, faithfulness check, LLM-as-a-judge
Stale knowledge base	Documents updated but embeddings not re-indexed	Delta re-embedding pipeline, document versioning, change detection
Multi-hop reasoning gap	Answer requires combining facts from 3+ chunks	Agentic RAG, iterative retrieval, GraphRAG for relational queries
Latency	Embedding + vector search + reranking + LLM call adds up	Caching, ANN index tuning, smaller retrieval models, async pipeline

Q8.2 [AI/ML Interview]

What happens when retrieval fails? How do you handle it?

Step 1 — Detect retrieval failure:

- Similarity score below threshold (e.g., cosine < 0.70) → flag as low-confidence retrieval
- Zero chunks returned above threshold → retrieval failure confirmed
- Retrieved chunks are semantically unrelated to query (LLM-as-a-judge check)

Step 2 — Recovery strategies:

- **Query reformulation** — rewrite query with different phrasing and retry retrieval
- **Fallback to keyword search** — BM25 as secondary retriever when dense retrieval fails
- **Expand scope** — retrieve from broader metadata category if narrow filter returned nothing
- **Graceful fallback** — return 'I don't have enough information to answer this reliably' rather than hallucinating

Step 3 — In LangGraph specifically:

Conditional edge: if retrieved_docs=[] → route to fallback_node (not generation_node). State carries the failure signal. Never pass empty context to the LLM.

Q8.3 [AI/ML Interview]

When would you use RAG instead of fine-tuning?

Dimension	Use RAG	Use Fine-tuning
Knowledge type	External, domain-specific, frequently updated documents	Behaviour, tone, format, style that doesn't change
Update frequency	Knowledge changes weekly/monthly	Knowledge is stable for months/years
Cost	No training cost — only inference + vector DB	Training compute cost + data curation cost
Transparency	Can cite sources — user can verify	Knowledge baked into weights — not explainable
Hallucination risk	Lower — grounded in retrieved facts	Higher on out-of-distribution queries
Latency	Higher — retrieval adds latency	Lower — just inference, no retrieval step
Best for	Enterprise knowledge base, legal/policy Q&A, product docs	Instruction following, specific output format, domain jargon

Real answer: In most enterprise GenAI projects, RAG first. Fine-tune only when you need the model to behave differently — not just to know different things. Often the best system uses both: RAG for knowledge, fine-tuning for format and style.

Q8.4 [AI/ML Interview]

Why use LangGraph for an AI workflow? Why not plain LangChain chains?

	LangChain Chains	LangGraph
Flow type	Linear / sequential	Graph — cycles, branches, parallel nodes
Conditionals	Limited — hard to branch on runtime state	First-class — conditional edges on any state value
Loops / iteration	Not supported natively	Natively supports cycles (e.g., retry until quality passes)
State persistence	No built-in checkpointing	MemorySaver / SqliteSaver — resume from any node
Multi-agent	Awkward — agents are chained, not orchestrated	Natural — each agent is a node; supervisor is the router
Partial failure recovery	Restart from beginning	Restart from last checkpoint node
Observability	Harder to trace which step failed	Each node is a traced unit — clear failure attribution

One-sentence answer: 'LangGraph gives you stateful, cyclical, recoverable agentic workflows. LangChain chains give you stateless linear pipelines. Any real-world agent needs LangGraph.'

Q8.5 [AI/ML Interview]

How would you handle failures in an agentic workflow?

Already covered in Q9 (Section 1) and Q6.2 (Section 6). Summary of the 5-layer strategy:

1. **Checkpointing** — persist state after each node (MemorySaver). Resume, don't restart.
2. **Node-level try/except** — catch at node boundary, write error to state, don't crash graph.
3. **Conditional edges** — route to fallback node on error state.
4. **Retry with backoff** — tenacity on LLM/tool calls for transient failures.
5. **Dead-letter / human review** — after N retries, escalate to human rather than loop forever.

New angle for this interview: Also mention **partial failure** — if one branch of a parallel graph fails, the healthy branches should complete. LangGraph's state model supports this: healthy nodes write their results; failed node writes error state. Downstream nodes check for error before using upstream results.

Part B — ML Models: Decision Trees, Random Forest, XGBoost

Q8.6 [AI/ML Interview]

How does a Decision Tree work?

A Decision Tree learns a hierarchy of if/else rules from data. At each node, it finds the feature and threshold that best splits the data — 'best' measured by reducing impurity.

Concept	Detail
Splitting criterion	Classification: Gini impurity or Entropy (information gain). Regression: MSE reduction.
Gini impurity	$Gini = 1 - \sum(p_i^2)$. Zero = perfectly pure node (all one class). Split that maximises purity reduction is chosen.
Tree depth	Deep tree = overfits (memorises training data). Shallow tree = underfits. Control with <code>max_depth</code> , <code>min_samples_split</code> .
Pruning	Post-pruning: grow full tree, then remove branches that don't improve validation performance. Reduces overfitting.
Strengths	Interpretable, handles mixed feature types, no scaling needed, fast prediction.
Weaknesses	High variance — small data changes → very different tree. Prone to overfitting without pruning. Poor on linear relationships.

Q8.7 [AI/ML Interview]

Why XGBoost instead of Random Forest?

Dimension	Random Forest	XGBoost
Ensemble type	Bagging — trains trees in parallel on bootstrap samples	Boosting — trains trees sequentially, each correcting previous errors
How it learns	Average of many independent trees (reduces variance)	Additive model — each tree fits residuals of previous trees (reduces bias)
Performance	Robust, rarely overfits, great default	Usually higher accuracy on tabular data — Kaggle competitions standard
Speed	Fast — trees in parallel	Slower to train (sequential), but optimised C++ implementation makes it fast in practice
Regularisation	Limited — <code>max_depth</code> , <code>n_estimators</code>	L1 + L2 regularisation, <code>learning_rate</code> , subsampling — better overfitting control
Missing values	Requires imputation	Handles missing values natively
Interpretability	Feature importance (mean impurity decrease)	Feature importance + SHAP values — more interpretable than RF
When RF is better	When you need speed, robustness, and don't want to tune many hyperparameters	When you need maximum predictive performance and are willing to tune

Interview answer: 'We chose XGBoost because our dataset had imbalanced classes and XGBoost's built-in `scale_pos_weight` parameter directly addresses that. It also gave us SHAP-based explainability which the client required. Random Forest would have been a valid baseline, but XGBoost gave measurably better F1 on our validation set.'

■ Model Choice Justification Formula

Always justify model choice with: performance metric, a specific feature need (e.g. missing value handling, SHAP), and the business context (interpretability, latency). Never say 'XGBoost is just better'.

Q8.8 [AI/ML Interview]

How do you handle overfitting?

Overfitting = model performs well on train but poorly on validation/test.

Technique	How it helps	Where to apply
Reduce model complexity	Fewer parameters → less capacity to memorise noise	Decision Tree: <code>max_depth</code> . XGBoost: <code>max_depth</code> , <code>n_estimators</code> .
Regularisation (L1/L2)	Penalises large weights, forces model to generalise	XGBoost: <code>reg_alpha</code> (L1), <code>reg_lambda</code> (L2). Neural nets: weight decay.
Early stopping	Stop training when validation loss stops improving	XGBoost: <code>early_stopping_rounds</code> . Neural nets: <code>EarlyStopping</code> callback.
Cross-validation	Evaluate on multiple held-out folds — robust measure of generalisation	Use 5-fold stratified CV for classification tasks.
More training data	More diverse examples reduce overfitting	Data augmentation if real data is scarce.
Dropout	Randomly disable neurons during training — forces redundancy	Neural nets / deep learning only.
Feature selection	Remove noisy or correlated features that add variance	Use SHAP values or feature importance to identify and drop.
Subsample / colsample	XGBoost-specific: train each tree on a random subset of rows/features	XGBoost: <code>subsample=0.8</code> , <code>colsample_bytree=0.8</code> is a good start.

Q8.9 [AI/ML Interview]

How do you evaluate the model?

Evaluation framework: Offline → Threshold tuning → Online

Metric	Use when	Formula / Key point
Accuracy	Balanced classes, equal cost of errors	$(TP+TN)/(Total)$. Misleading on imbalanced datasets.
Precision	Cost of false positive is high (spam filter)	$TP/(TP+FP)$. Of predicted positives, how many are real?
Recall	Cost of false negative is high (fraud, cancer)	$TP/(TP+FN)$. Of actual positives, how many did we catch?
F1 Score	Imbalanced classes, need balance of P and R	$2 \cdot P \cdot R / (P+R)$. Harmonic mean — punishes extreme imbalance.
ROC-AUC	Compare models across all thresholds	Area under TPR vs FPR curve. 0.5=random, 1.0=perfect.
PR-AUC	Highly imbalanced datasets (ROC-AUC is optimistic)	Precision-Recall curve AUC. More informative than ROC-AUC when positives are rare.
Log loss	Probabilistic classifiers — measures calibration	Lower = better calibrated probability estimates.
RMSE / MAE	Regression tasks	RMSE penalises large errors more. MAE is more robust to outliers.

Always say: 'I chose [metric] because in this problem, the cost of [false positive / false negative] is [higher/lower], so optimising for [precision/recall] was more aligned with the business outcome.'

Q8.10 [AI/ML Interview]

How do you handle imbalanced data?

Three levels of intervention: data level → algorithm level → evaluation level

Data level:

- **Oversampling minority class** — SMOTE (Synthetic Minority Oversampling): generates synthetic samples by interpolating between existing minority samples. Better than simple duplication.
- **Undersampling majority class** — randomly remove majority samples. Risk: losing information. Use with large datasets.
- **Combined** — SMOTE + Tomek Links or SMOTE + ENN for cleaner decision boundaries.

Algorithm level:

- **class_weight='balanced'** — scikit-learn automatically adjusts weights inversely proportional to class frequency. Works for most models.

- **scale_pos_weight (XGBoost)** — set to count(negative)/count(positive). Tells XGBoost to penalise minority class misclassification more.
- **Threshold tuning** — lower the classification threshold below 0.5 to increase recall on minority class. Plot precision-recall curve to choose optimal threshold.

Evaluation level:

- Never use accuracy on imbalanced data. Use F1, PR-AUC, or ROC-AUC.
- Stratified K-fold cross-validation — ensures each fold preserves class ratio.

Part C — OCR: Accuracy, Errors, Evaluation, and Improvement

Q8.11 [AI/ML Interview]

How is OCR accuracy measured?

Metric	Definition	When to use
Character Error Rate (CER)	$CER = (S + D + I) / N$ where S=substitutions, D=deletions, I=insertions, N=total characters in ground truth. Lower = better.	Fine-grained accuracy, character-level tasks
Word Error Rate (WER)	Same formula but at word level. $WER = (S + D + I) / total_words$.	NLP downstream tasks where word integrity matters
Field Accuracy	For structured docs (forms, invoices): % of key fields extracted correctly. A field is correct only if the full value matches.	Document extraction — most business-relevant metric
Page-level accuracy	% of pages where all extracted fields are correct. Harsh but reflects production quality.	Enterprise document processing SLAs
Confidence score distribution	OCR engines return confidence per character/word. Track % of tokens above confidence threshold.	Monitoring and flagging low-confidence extractions for human review

Interview answer structure: 'We measured OCR accuracy at three levels: CER for character-level quality, field-level accuracy for business correctness, and confidence score distribution for production monitoring. A page was marked as passing only if all required fields were extracted correctly above 95% confidence.'

Q8.12 [AI/ML Interview]

What can cause OCR errors?

Error Source	Examples	Signal
Image quality	Low DPI, blur, skew, noise, shadows	CER spikes on specific document batches
Font / layout complexity	Unusual fonts, overlapping text, tables, multi-column	Errors clustered on specific document types
Handwriting	Mixed print/handwritten, cursive	Low confidence scores on affected regions
Language / script	Mixed languages, special characters, abbreviations	Character substitution errors (e.g., '1' vs 'l', '0' vs 'O')
Scanning artefacts	Stamps, watermarks, folds, torn edges	Errors in consistent regions of documents
Low contrast	Light ink on light background, coloured forms	Missed characters (deletions) rather than substitutions

Q8.13 [AI/ML Interview]

How can OCR performance be improved?

Pre-processing (improve input quality):

- **Image binarisation** — convert to black/white with adaptive thresholding (Otsu's method)
- **Deskewing** — detect and correct document rotation before OCR
- **Denoising** — Gaussian blur, median filter to remove noise while preserving edges
- **DPI upscaling** — resample to 300+ DPI if original is low-resolution
- **Contrast enhancement** — histogram equalisation for faded documents

OCR engine choices:

- **Tesseract** — open source, good for clean printed text, customisable with domain-specific training
- **Azure Document Intelligence** — best for structured forms/invoices, tables, mixed layouts
- **AWS Textract** — strong on tables and key-value pairs
- **Google Document AI** — strong on handwriting

Post-processing (improve output quality):

- **Dictionary / domain correction** — match extracted tokens to known vocabulary (e.g., valid drug names, company names)
- **Regex validation** — validate field formats (dates, PAN numbers, amounts) and flag mismatches

- **LLM-based correction** — use LLM to interpret and correct low-confidence OCR output with surrounding context
- **Confidence-based human review** — route low-confidence extractions to human reviewer queue

Q8.14 [AI/ML Interview]

How would you evaluate the OCR system on real-world documents?

Evaluation pipeline — 4 stages:

1. **Golden dataset** — manually annotated set of 200–500 representative real documents covering all document types, quality levels, and edge cases. Ground truth = human-verified extracted fields.
2. **Offline evaluation** — run OCR pipeline on golden dataset. Measure CER, WER, field accuracy, and confidence distribution. Segment results by document type and quality tier.
3. **Error analysis** — manually inspect top-30 failure cases. Cluster by error type (image quality, font, layout). Prioritise fixes by frequency × business impact.
4. **Production monitoring** — log confidence scores per field for every document processed. Alert when: average confidence drops below threshold, field extraction failure rate rises, human review queue grows. Use these signals as real-world accuracy proxy.

Closing answer: 'We treat OCR evaluation like any ML evaluation — golden dataset for offline, confidence monitoring for online, and periodic re-annotation as new document types are encountered in production.'

Section 8 Key Takeaway — The 3-Question Interview Arc

Stage	Question	What to prepare
1	What did you build?	Project in 4 layers: Problem → Architecture → Components → Results
2	Why did you use this technology?	Decision justification: metric that drove the choice + alternatives considered + trade-offs accepted
3	How did you measure the result?	Metrics chosen + baseline vs result + production monitoring approach

Knowing a technology is only one part. The interview moves fast from 'what' to 'why' to 'how did you measure it' — have all three answers ready for every major component in your project.

SECTION 9 — Agentic AI Engineer | Financial Services | Inference · KV Cache · Distillation · RAG Ops

Most advanced interview set in this guide. Focuses on production LLM serving: inference optimization, KV cache internals, HPA scaling, distillation, quantization, RAG document versioning, table ingestion, and LangGraph code. These are senior/staff engineer questions.

Part A — LLM Inference & Optimization

Q9.1 [FinServ Agentic AI]

How did you develop an LLM inference microservice? What inference optimization techniques did you use?

LLM inference microservice — standard architecture:

- **Serving framework** — vLLM (self-hosted) or Azure OpenAI endpoint. vLLM uses PagedAttention for efficient KV cache memory management.
- **API layer** — FastAPI async endpoints. POST /v1/chat/completions (OpenAI-compatible). Health check + readiness endpoints for Kubernetes.
- **Request queue** — async queue absorbs traffic spikes. Prevents GPU starvation from bursty load.
- **Containerisation** — Docker image with CUDA drivers. Deployed on GPU node pool in AKS/GKE.

Key inference optimization techniques:

Technique	What it does	Gain
KV Cache	Cache Key/Value tensors from previous tokens — avoid recomputation	2–10× throughput on long contexts
Continuous batching	Add new requests to a running batch mid-generation instead of waiting for full batch to complete	3–5× GPU utilisation improvement
PagedAttention (vLLM)	Manage KV cache in non-contiguous memory pages — eliminates fragmentation	Enables 2–4× more concurrent requests
Quantisation (INT8/INT4)	Reduce weight precision — smaller memory footprint, faster compute	2–4× memory reduction, ~1.5× speed
Speculative decoding	Small draft model proposes tokens, large model verifies in parallel	2–3× latency reduction for long outputs
Flash Attention	Reorders attention computation to minimise memory I/O	2–4× attention speed, 10–20× memory reduction
Tensor parallelism	Split model weights across multiple GPUs	Enables models too large for single GPU
Prompt caching	Cache KV state of repeated system prompt prefix across requests	Reduces latency and cost for common system prompts

Q9.2 [FinServ Agentic AI]

How does KV Cache work and how can it be managed?

How KV Cache works:

In transformer inference, every token attends to all previous tokens using Key (K) and Value (V) matrices. Without caching, these K and V tensors are recomputed for every token at every step — $O(n^2)$ cost per sequence.

KV Cache stores the computed K/V tensors for all previous tokens. At each new generation step, only the new token's K/V is computed and appended. Reduces computation from $O(n^2)$ to $O(n)$.

Memory cost:

KV cache size = $2 \times \text{num_layers} \times \text{num_heads} \times \text{head_dim} \times \text{sequence_length} \times \text{batch_size} \times \text{dtype_bytes}$.

For GPT-3 (96 layers, 96 heads, 128 head_dim): ~1GB per 1000 tokens at float16. At 8k context and batch=32 → ~256GB. This is the primary memory bottleneck at scale.

KV Cache management strategies:

Strategy	How it works	Trade-off
PagedAttention (vLLM)	Allocate KV cache in fixed-size pages (like OS virtual memory). Pages assigned on demand, freed when sequence completes.	Near-zero fragmentation. Current production standard.
KV cache eviction	Evict KV cache of completed sequences immediately. LRU eviction under memory pressure.	Increases latency for evicted sequences if they need to be recomputed.
Sliding window attention	Only cache the last W tokens instead of the full history. Used in Mistral.	Loses context beyond window. Acceptable for tasks where distant context is less relevant.
Prompt caching	Cache KV state of a fixed system prompt prefix. Reuse across all requests sharing that prefix.	Only works when prefix is identical across requests. Reduces TTFT significantly.
Quantised KV cache	Store KV tensors in INT8 instead of FP16.	50% memory saving with small quality loss. Used when GPU VRAM is constrained.

■ FinServ Scale Pattern

Financial services question pattern: 'We have 10,000 concurrent users hitting our LLM endpoint. How do you manage KV cache?' Answer: vLLM with PagedAttention + INT8 KV cache quantisation + sliding window attention if context > 8k + prompt caching for fixed system prompt prefix.

Q9.3 [FinServ Agentic AI]

Inference-serving parameters: number of sequences and batched tokens. How do you configure them?

Parameter	What it controls	How to tune
max_num_seqs	Maximum concurrent sequences in a single batch. Higher = better GPU utilisation but more KV cache memory.	Start with GPU_VRAM / (avg_KV_cache_per_seq). Monitor OOM errors and reduce if hit.
max_num_batched_tokens	Maximum total tokens across all sequences in one forward pass. Controls compute per step.	Set to GPU compute capacity. Typical: 4096–32768. Higher = better throughput, higher latency per step.
max_model_len	Maximum context length the server will accept.	Set to model's max context. Longer sequences consume more KV cache per slot.
gpu_memory_utilisation	Fraction of GPU VRAM allocated to KV cache (vLLM). Rest holds model weights.	Start at 0.90. Reduce if OOM on large batches.
tensor_parallel_size	Number of GPUs to shard model across.	1 for models fitting in single GPU. 2, 4, 8 for larger models.
dtype	Weight precision: float16, bfloat16, int8, int4.	bfloat16 for most GPU types. int8/int4 when VRAM is the bottleneck.

Latency vs throughput trade-off: Higher batch size → higher throughput (requests/sec), higher latency per request. For interactive use cases (chatbot), optimise for TTFT (time to first token). For batch processing, optimise for throughput.

Q9.4 [FinServ Agentic AI]

Setting up HPA (Horizontal Pod Autoscaler) for LLM serving — what parameters do you use?

Problem with default HPA for LLMs: Default HPA scales on CPU/memory. LLM serving is GPU-bound and queue-depth-bound — CPU stays low even when the model is fully saturated. CPU-based HPA will not scale when needed.

Correct approach — custom metrics HPA:

```
# HPA targeting GPU queue depth custom metric
apiVersion: autoscaling/v2
kind: HorizontalPodAutoscaler
spec:
  minReplicas: 2
  maxReplicas: 10
  metrics:
  - type: External
  external:
    metric:
      name: llm_pending_requests # from Prometheus
      target:
        type: AverageValue
        averageValue: 10 # scale when >10 pending per pod
```

Scaling Metric	When to use	Threshold guidance
pending_requests (queue depth)	Best signal for LLM serving — directly measures backpressure	Scale out when > 5–10 pending requests per pod
GPU utilisation (%)	Good secondary metric — confirms GPU is the bottleneck	Scale out when GPU util > 80% for > 2 minutes
p95 request latency	User-experience driven scaling	Scale out when p95 TTFT > SLA (e.g. > 2 seconds)
tokens_generated/sec	Throughput-based scaling for batch workloads	Scale out when throughput falls below target

Additional HPA considerations: Set `scaleDown stabilisationWindow` (e.g. 300s) to prevent thrashing. Pre-warm GPU pods — cold start for LLM pods is 2–5 minutes (model loading). Consider Karpenter/KEDA for faster node provisioning.

Q9.5 [FinServ Agentic AI]

LLM-as-a-Judge — how to select representative examples for evaluation?

LLM-as-a-judge uses a capable LLM (e.g. GPT-4o) to score another model's outputs on a rubric. The quality of the evaluation depends entirely on the quality of the example set.

How to select representative examples:

- Stratify by query type** — cluster your production queries (k-means on query embeddings). Sample from each cluster to ensure coverage of all query intents, not just the most common ones.
- Include hard cases** — deliberately include queries where the model is known to struggle: multi-hop questions, ambiguous queries, out-of-domain queries, adversarial inputs.
- Include boundary cases** — queries near the decision boundary between 'good' and 'bad' answers. These have the most signal for discrimination.

4. **Balance by difficulty** — roughly 60% easy (should always pass), 30% medium, 10% hard (edge cases). Prevents floor/ceiling effects in scoring.
5. **Minimum size** — 100–200 examples for reliable eval signal. Below 50, variance is too high to detect regressions.
6. **Refresh periodically** — real user queries shift over time (query drift). Re-sample from recent production traffic quarterly.

Judge calibration: Before using LLM-as-a-judge at scale, validate it against human judgements on 50 examples. If judge-human agreement > 80%, it is reliable for automated use.

Part B — Model Distillation & Quantization

Q9.6 [FinServ Agentic AI]

What is model distillation? Advantages and disadvantages?

Knowledge Distillation: Train a smaller 'student' model to mimic the output distribution (soft labels) of a larger 'teacher' model — not just the hard class labels, but the full probability distribution over outputs.

How it works:

Teacher produces soft probability outputs (e.g. 70% class A, 20% class B, 10% class C). Student is trained to match this distribution using KL divergence loss in addition to standard cross-entropy on hard labels. The soft distribution carries richer information than hard labels (it encodes the teacher's 'uncertainty').

Aspect	Detail
Types	Response distillation: match final outputs. Feature distillation: match intermediate layer activations. Data-free distillation: use teacher to generate synthetic training data for student.
LLM-specific	GPT-4 → GPT-3.5-level student via distillation. Phi-2, Phi-3 (Microsoft) are heavily distilled from larger models. DistilBERT = 40% smaller, 97% of BERT performance.
Advantages	Student is 2–10× smaller and faster. Retains most of teacher's performance. Lower inference cost and latency. Can run on edge/constrained hardware.
Disadvantages	Requires access to teacher model outputs (not always possible). Training cost — still need to fine-tune the student. Performance gap widens on complex reasoning tasks. Teacher's errors are also distilled.
vs Fine-tuning	Distillation: compress a large model's behaviour into a smaller one. Fine-tuning: adapt an existing model to a new task/domain. Different goals, often combined.

Q9.7 [FinServ Agentic AI]**What is model quantization? Advantages and disadvantages?**

Quantization: Reduce the numerical precision of model weights (and optionally activations) from FP32/FP16 to INT8, INT4, or lower. Trades a small amount of accuracy for significant memory and speed gains.

Type	What is quantized	Memory saving	Quality impact
Post-Training Quantization (PTQ)	Weights only, after training is complete. No retraining needed.	2× (INT8), 4× (INT4)	Small — < 1% accuracy drop for INT8. 1–3% for INT4.
Quantization-Aware Training (QAT)	Simulate quantization during training — model learns to compensate.	2–4×	Minimal — recovers most PTQ quality loss. Requires retraining.
GGUF / GGML (llama.cpp)	INT4/INT8 weights, CPU-friendly format.	4×	Acceptable for 7B–13B models. Larger quality drop on smaller models.
AWQ (Activation-aware Weight Quantization)	INT4 with activation-aware scaling — protects salient weights.	4×	Better than naive INT4. Near INT8 quality.
GPTQ	INT4, post-training, layer-by-layer optimisation.	4×	Very good — near-lossless for large models.

	Advantages	Disadvantages
Quantization	2–4× memory reduction. 1.5–2× speed increase. Enables larger models on same GPU. Reduces serving cost.	Quality degrades on complex reasoning tasks. INT4 can produce gibberish on edge cases. Requires calibration dataset for best results. Not all hardware supports INT4 natively.

■ Production Quantization Answer

Interview answer: 'For our financial services LLM endpoint, we used INT8 PTQ via bitsandbytes. It gave us 2× memory reduction with < 1% accuracy drop on our evaluation set, enabling us to double the number of concurrent requests on the same GPU fleet.'

Q9.8 [FinServ Agentic AI]**Why do LLMs sometimes repeat the same sequence? How do you mitigate it?****Root causes of repetition:**

- **High probability loops** — once a token sequence is generated, it conditions future tokens. If that sequence has high probability in the training distribution, the model can get 'stuck' in a loop.

- **Low temperature + greedy decoding** — deterministic generation amplifies repetition. The most probable token at each step reinforces the pattern.
- **Insufficient training diversity** — if training data contained repetitive text, the model learned to replicate it.
- **Long context degradation** — attention mechanisms lose focus over very long sequences, causing the model to 'fall back' to recently generated text.

Mitigation strategies:

Strategy	How it works	Trade-off
Repetition penalty	Multiply logit probability of already-generated tokens by a penalty factor < 1. Makes recently seen tokens less likely to be selected again.	Too high → incoherent output (avoids necessary repetition like pronouns)
No-repeat n-gram	Track all n-grams generated so far. Set probability of generating any seen n-gram to 0.	Can block legitimate repetition. Works best for n≥3.
Frequency penalty (OpenAI)	Reduce logit of tokens proportionally to how often they've appeared in the output so far.	Cumulative effect — useful for long outputs.
Presence penalty (OpenAI)	Binary: reduce logit of any token that has appeared at all, regardless of frequency.	Encourages diversity of topics, not just words.
Temperature / top-p increase	Introducing randomness breaks deterministic loops.	Higher temperature can hurt coherence and factual accuracy.
Stop sequences	Define explicit stop tokens to halt generation before a loop deepens.	Only catches loops at known boundaries.

Q9.9 [FinServ Agentic AI]

Handling hallucinations in LLM applications — full approach.

Three-layer hallucination defence:

Layer 1 — Prevention (before generation):

- RAG grounding — retrieve and inject relevant facts before generation
- Explicit system prompt constraint: 'Answer ONLY using the provided context. If the information is not present, say I don't know.'
- Reduce temperature — lower temperature = more conservative, less inventive
- Retrieval quality — higher quality retrieved context → less need for model to 'fill in'

Layer 2 — Detection (after generation, before response):

- **NLI-based check** — does the retrieved context entail the generated answer? (NLI = Natural Language Inference)
- **LLM-as-a-judge faithfulness check** — 'Is every claim in this answer supported by the provided context? Respond YES/NO with reasons.'
- **Self-consistency** — generate N answers, check if they agree. Disagreement = hallucination signal.
- **Confidence score** — low token probability on specific claims signals potential hallucination

Layer 3 — Containment (if hallucination detected):

- Return safe fallback: 'I don't have sufficient information to answer this reliably.'
- Route to human review for high-stakes financial queries
- Log and flag for golden dataset update

Financial services specific: In FinServ, hallucinated figures (interest rates, regulatory thresholds, account balances) are compliance risks, not just UX issues. Every numerical claim should be verifiable against a retrieved source with a citation.

Part C — RAG Operations: Document Versioning & Table Handling

Q9.10 [FinServ Agentic AI]

Managing document version changes in RAG: section modifications, section removal, and maintaining index consistency.

The core problem: RAG indexes are a snapshot. When source documents change, stale chunks can surface outdated information — a serious risk in financial services (outdated regulations, superseded policies).

Architecture for document versioning:

- **Document registry** — maintain a metadata store (document_id, version, last_modified, checksum, section_ids). Compare checksums on each ingestion run to detect changes.
- **Chunk-to-document mapping** — every chunk in the vector DB stores its source document_id, section_id, and version. This enables targeted updates without full re-indexing.

Change Type	Detection	Update Strategy
Section-level modification	Checksum of section content changed	Delete all chunks with matching section_id from vector DB. Re-chunk and re-embed the modified section only. Update version metadata.
Complete section removal	Section_id exists in registry but not in new document	Hard delete all chunks with matching section_id. Remove section from registry. Tombstone record for audit trail.
New section added	Section_id not in registry	Chunk and embed new section. Insert into vector DB with new section_id. Add to registry.
Document replacement (full)	Document checksum changed	Delete all chunks with matching document_id. Re-ingest full document. More expensive — use only when section-level diffing is not feasible.

Consistency safeguards:

- **Transactional updates** — delete old chunks and insert new ones in a single atomic operation (or use versioned namespaces and swap). Prevents a window where both old and new chunks exist simultaneously.
- **Version field on every chunk** — at query time, filter to latest version. Allows rollback by querying an older version.
- **Scheduled consistency check** — nightly job compares document registry against vector DB chunk counts. Alerts on orphaned chunks or missing sections.

Q9.11 [FinServ Agentic AI]

Handling large tables in documents that span multiple pages — parsing and ingestion strategies.

Why standard chunking fails for tables: RecursiveCharacterTextSplitter splits on character count. A table row split mid-row loses all meaning. A header row separated from data rows makes the chunk uninterpretable.

Parsing strategies for multi-page tables:

- **Azure Document Intelligence / AWS Textract** — purpose-built for table detection. Identifies table boundaries, merges multi-page tables, returns structured row/cell data. Best for enterprise document pipelines.
- **pdfplumber** — open source, good for simple tables in text-based PDFs. Can detect table bounding boxes and extract cell-level content. Struggles with merged cells and complex layouts.
- **Camelot / Tabula** — specialised PDF table extractors. Camelot uses lattice (line-based) and stream (whitespace-based) modes. Good for well-structured financial tables.

- **Multi-page table stitching** — detect continuation headers ('Table 3 continued'), match schema of next-page table, merge before chunking.

Ingestion strategies after parsing:

Strategy	How it works	Best for
Row-level chunking	Each table row becomes one chunk. Metadata: table_name, column_headers, row_index, page_number.	Lookup tables, financial data tables where each row is independently queryable
Table-as-text serialisation	Convert table to natural language: 'In Q3 2024, revenue was \$4.2B, up 12% YoY.' Use LLM for serialisation.	Dense tables where narrative context aids retrieval
Table-as-JSON chunk	Store entire table as structured JSON chunk with metadata. Retrieve whole table when any cell matches.	Small-to-medium tables where full context is always needed
Summary + raw dual index	Generate a text summary of the table's purpose and key figures → embed summary for retrieval. Store raw table separately for answer generation.	Large complex tables — summary captures semantics, raw data enables precise answers

Column header preservation rule: Every chunk derived from a table **MUST** include the column headers in its metadata or text. A chunk containing '42.3 | 18.7 | 91.2' is useless without knowing those are 'Revenue | EBITDA | Total Assets'.

Part D — Evaluation Metrics: Micro F1 vs Macro F1

Q9.12 [FinServ Agentic AI]

Micro F1 vs Macro F1 — difference and when to use each?

Both are F1 averages across multiple classes, but they aggregate differently:

	Macro F1	Micro F1
How calculated	Calculate F1 per class independently. Average the per-class F1 scores.	Pool all TP, FP, FN across classes. Calculate one global F1 from the pooled counts.
Class weighting	Each class contributes equally regardless of size.	Each sample contributes equally — large classes dominate.
Sensitive to	Minority class performance — a poor score on a rare class drags down the average.	Overall accuracy on the dataset — dominated by the most common class.
Use when	All classes matter equally, especially small ones. Imbalanced datasets where minority class performance matters.	Class distribution reflects real-world importance. Large class performance should dominate. Balanced datasets.
Example	Medical diagnosis: rare disease (10 cases) should matter as much as common cold (10,000 cases). Use Macro.	Document classification: 90% of docs are class A. Getting class A right matters most. Use Micro.
Pitfall	Can be misleadingly low if one rare class has poor performance, even if all others are excellent.	Can be misleadingly high if the dominant class is easy, masking poor performance on rare classes.

Weighted F1: Third option — weighted average of per-class F1 by class support (number of true instances). Compromise between Macro and Micro. Use when classes have different sizes but you want class size to influence the average proportionally.

Financial services answer: 'In our fraud detection model with heavy class imbalance (0.1% fraud rate), we use Macro F1 to ensure the fraud class F1 is given equal weight. Micro F1 would be near-perfect even if our model detected no fraud at all.'

Part E — Round 2: LangGraph Summarizer Agent (Working Code)

Q9.13 [FinServ Round 2]

Write a working LangGraph summarizer agent.

Part 1 — Imports, state, and nodes:

```

from typing import TypedDict, List, Annotated
from langgraph.graph import StateGraph, END
from langchain_openai import AzureChatOpenAI
from langchain_core.documents import Document
import operator

class SummaryState(TypedDict):
    documents: List[Document]
    chunk_summaries: Annotated[List[str], operator.add]
    final_summary: str
    error: str

llm = AzureChatOpenAI(model="gpt-4o", temperature=0)

def map_node(state: SummaryState):
    try:
        summaries = []
        for doc in state["documents"]:
            r = llm.invoke([
                {"role": "system", "content": "Summarise concisely."},
                {"role": "user", "content": doc.page_content}])
            summaries.append(r.content)
        return {"chunk_summaries": summaries}
    except Exception as e:
        return {"error": str(e)}

def reduce_node(state: SummaryState):
    combined = " ".join(state["chunk_summaries"])
    r = llm.invoke([
        {"role": "system", "content": "Combine into one summary."},
        {"role": "user", "content": combined}])
    return {"final_summary": r.content}

def should_reduce(state) -> str:
    return "error_end" if state.get("error") else "reduce"

```

Part 2 — Graph assembly and run:

```

graph = StateGraph(SummaryState)
graph.add_node("map", map_node)
graph.add_node("reduce", reduce_node)
graph.set_entry_point("map")
graph.add_conditional_edges("map", should_reduce, {
    "reduce": "reduce",
    "error_end": END
})
graph.add_edge("reduce", END)
app = graph.compile()

docs = [Document(page_content="First chunk..."),
        Document(page_content="Second chunk...")]
result = app.invoke({
    "documents": docs, "chunk_summaries": [],
    "final_summary": "", "error": ""})
print(result["final_summary"])

```

Key points for the interview:

- * **Annotated[`List[str]`, `operator.add`]** — appends to list across node calls instead of overwriting.
- * **Conditional edge** — `should_reduce` checks error state before routing. Prevents calling reduce with empty summaries.
- * **Map-Reduce pattern** — map runs per document (parallelisable). Reduce synthesises all outputs.
- * **Extension** — add `MemorySaver` for checkpointing. Add validation node after reduce. Wrap map with tenacity retry.

Q9.14 [FinServ Round 2]**What latency metrics should be measured for an LLM serving endpoint and why?**

Metric	Definition	Why it matters
TTFT (Time to First Token)	Time from request receipt to first token streamed to client.	Primary user-experience metric for interactive chat. Users perceive TTFT as 'responsiveness'. Target: < 500ms.
TBT (Time Between Tokens)	Average interval between consecutive streamed tokens.	Determines perceived streaming smoothness. High TBT causes stuttering. Target: < 50ms.
E2EL (End-to-End Latency)	Total time from request to complete response.	Relevant for batch/async workloads. Not useful for streaming UX.
TPOT (Time Per Output Token)	E2EL / number of output tokens generated.	Normalised throughput metric. Enables fair comparison across responses of different lengths.
Throughput (tokens/sec)	Total tokens generated per second across all concurrent requests.	GPU efficiency metric. Low throughput = underutilised GPU or batching problem.
Queue wait time	Time request spends in queue before GPU starts processing.	Diagnoses capacity issues. High queue wait = need more replicas or better batching.
p50 / p95 / p99 latency	Percentile latency distribution.	p50 = typical case. p95/p99 = tail latency — what the worst 5%/1% of users experience. SLAs should be on p95.

Financial services SLA example: TTFT p95 < 1s, E2EL p95 < 10s for responses up to 500 tokens, throughput > 500 tokens/sec per pod, queue wait p99 < 2s.

Section 9 Key Takeaway — Senior AI Engineer Signal

This interview tested three things simultaneously: (1) can you build and operate LLM serving at scale?, (2) do you understand the full lifecycle — from inference optimisation to document versioning to evaluation?, (3) can you write production-quality LangGraph code under pressure?

Signal	How to demonstrate it
Inference depth	Know KV cache internals, PagedAttention, quantization trade-offs — not just 'use vLLM'
Operational thinking	Every answer should include: how you'd monitor it, what metric proves it's working, what breaks at scale
RAG maturity	Document versioning, table handling, and consistency management show production RAG experience beyond basic retrieval
Code quality	LangGraph code should include state typing, error handling, conditional edges, and clear node responsibilities

GenAI Interview Master Guide — Compiled by Claude for Narayanan | Accenture | September 2026